

MailForge

A Modern Mail Server in C++20

Version: 1.0

1. Project Overview

Objective

Develop a production-inspired mail server in modern C++ that implements core email protocols and demonstrates software engineering, networking, concurrent programming, security, and systems design.

The project is intended as a portfolio-quality application suitable for technical interviews and campus placements.

This is **not** intended to replace enterprise mail servers such as Postfix or Exim. Instead, it aims to showcase the engineering skills required to build similar systems.

2. Motivation

The project demonstrates proficiency in:

- Modern C++
- Operating Systems
- Computer Networks
- Concurrent Programming
- Protocol Design
- Secure Authentication
- Software Architecture
- Database Design
- Performance Engineering
- Testing and Documentation

Unlike typical CRUD applications, this project involves implementing networking protocols, designing state machines, and building modular infrastructure.

3. Project Goals

Functional Goals

- Accept email using SMTP
- Store emails securely
- Manage users
- Authenticate clients
- Retrieve emails
- Deliver emails between servers
- Support encrypted communication
- Maintain delivery queues
- Generate logs

Non-Functional Goals

- Modular architecture
 - High performance
 - Thread-safe implementation
 - Easy to extend
 - Unit tested
 - Dockerized
 - Cross-platform (Linux first)
-

4. Technology Stack

Language

- C++20

Build System

- CMake

Compiler

- GCC
- Clang

Networking

- Boost.Asio

Cryptography

- OpenSSL

Database

- SQLite

Logging

- spdlog

Configuration

- JSON
- nlohmann/json

Testing

- GoogleTest

Formatting

- clang-format

Static Analysis

- clang-tidy

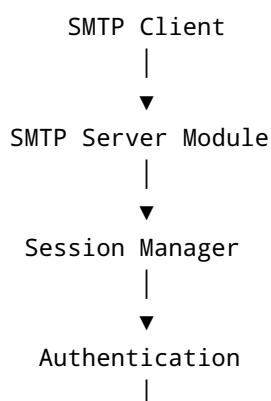
Version Control

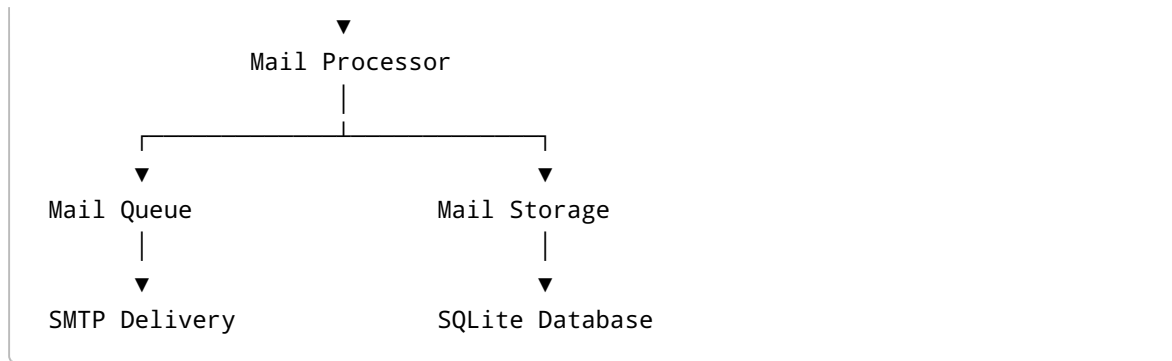
- Git
- GitHub

Containerization

- Docker

5. System Architecture





6. High-Level Modules

Network Layer

Responsibilities

- TCP server
- Accept connections
- Session creation
- Async I/O

SMTP Module

Responsibilities

- SMTP command parser
- SMTP state machine
- Email validation
- DATA handling

Authentication Module

Responsibilities

- User login
- Password verification
- Session authorization

Mail Storage Module

Responsibilities

- Store emails

- Retrieve emails
 - Delete emails
 - Folder management
-

Queue Module

Responsibilities

- Pending deliveries
 - Retry mechanism
 - Worker threads
-

Database Module

Responsibilities

- Users
 - Messages
 - Attachments
 - Metadata
-

Logging Module

Responsibilities

- Request logging
 - Error logging
 - Audit logs
-

Configuration Module

Responsibilities

- Load configuration
 - Runtime validation
 - Defaults
-

Utilities

Responsibilities

- Base64
- MIME helpers

- Time utilities
 - File utilities
-

7. Directory Structure

MailForge/

docs/
architecture/
benchmarks/

include/

network/
smtp/
auth/
storage/
database/
queue/
config/
logging/
utils/
mime/
security/

src/

network/
smtp/
auth/
storage/
database/
queue/
config/
logging/
utils/
mime/
security/

tests/

config/

scripts/

logs/

mail/

```
docker/  
  
benchmark/  
  
assets/  
  
README.md  
LICENSE  
CMakeLists.txt
```

8. Proposed Class Structure

```
Application  
  
Server  
  
TcpListener  
  
ClientSession  
  
SMTPParser  
  
SMTPStateMachine  
  
MailMessage  
  
Mailbox  
  
DatabaseManager  
  
Authenticator  
  
QueueManager  
  
WorkerPool  
  
Logger  
  
ConfigManager
```

9. Database Design

Users

```
id
username
password_hash
created_at
```

Messages

```
id
sender
recipient
subject
body
timestamp
status
```

Attachments

```
id
message_id
filename
size
path
```

10. Configuration File

Example

```
{
  "port":2525,
  "thread_pool":8,
  "database":"mail.db",
  "mail_directory":"mail/",
  "log_level":"info",
```



```
"max_connections":500
}
```

11. Development Phases

Phase 1

Project setup

Deliverables

- Repository
- CMake
- Logging
- Configuration
- Documentation

Estimated time

3–4 days

Phase 2

Networking

Deliverables

- TCP server
- Async connections
- Session handling
- Thread pool

Estimated time

1 week

Phase 3

SMTP

Deliverables

- SMTP parser
- SMTP state machine
- HELO

- EHLO
- MAIL FROM
- RCPT TO
- DATA
- QUIT

Estimated time

2 weeks

Phase 4

Filesystem Mail Storage

Deliverables

- Store mail
- Read mail
- Mail directories

Estimated time

1 week

Phase 5

Authentication

Deliverables

- User database
- Password hashing
- AUTH LOGIN

Estimated time

1 week

Phase 6

SQLite Integration

Deliverables

- User tables
- Message tables

- Queries
- Transactions

Estimated time

1 week

Phase 7

Mail Queue

Deliverables

- Queue manager
- Retry logic
- Worker threads

Estimated time

1 week

Phase 8

TLS

Deliverables

- STARTTLS
- OpenSSL integration

Estimated time

1 week

Phase 9

POP3

Deliverables

- LIST
- RETR
- DELE

Estimated time

2 weeks

Phase 10

SMTP Client

Deliverables

- Outgoing delivery
- DNS lookup
- Retry

Estimated time

2 weeks

Phase 11

MIME

Deliverables

- HTML mail
- Attachments
- Base64
- Multipart parsing

Estimated time

2 weeks

Phase 12

Testing and Benchmarking

Deliverables

- Unit tests
- Integration tests
- Load testing
- Benchmark report

Estimated time

2 weeks

12. Testing Strategy

Unit Tests

- SMTP parser
- Queue
- Database
- Authentication
- MIME

Integration Tests

- SMTP session
- Mail delivery
- Mail retrieval

Performance Tests

- 100 clients
- 500 clients
- 1000 clients

Stress Tests

- Invalid packets
 - Flood connections
 - Concurrent login
-

13. Documentation

The repository should include:

- README
 - Build guide
 - Architecture guide
 - Module documentation
 - API documentation
 - Sequence diagrams
 - Benchmarks
 - Future roadmap
-

14. Frontend Decision

A frontend is **not required** to demonstrate a mail server.

Core Project (Resume Version)

- SMTP server
- POP3/IMAP server
- Authentication
- Queue
- SQLite
- TLS
- Logging
- Docker
- Tests

This alone is enough for a strong systems project.

Optional Frontend (Later)

After the backend is complete, a lightweight web interface can be added for administration.

Features:

- Login
- Inbox
- Sent Mail
- Compose Mail
- Queue Monitoring
- Server Statistics
- Log Viewer

A minimal frontend built with HTML/CSS/JavaScript or a small React application is sufficient because its purpose is to demonstrate the backend, not replace a full webmail client.

15. Future Enhancements

- IMAP support
 - DKIM
 - SPF
 - DMARC
 - Spam filtering
 - Full-text email search
 - REST API
 - Kubernetes deployment
 - Prometheus metrics
 - Grafana dashboards
 - Distributed mail storage
 - Message replication
 - Plugin architecture
 - OAuth2 authentication
 - WebSocket notifications
-

16. Resume Value

This project showcases:

- Modern C++ software engineering
- Network protocol implementation
- Concurrent systems programming
- Secure authentication
- Database integration
- Software architecture
- Testing and benchmarking
- Production-style development practices

It is intended to be the flagship project in a systems/backend engineering portfolio.